02613 Python and High Performance Computing

Answers for Week 1

---

1. Connecting and transferring files to the HPC

The focus of this exercise is to learn how to connect to the HPC nodes, transfer files and run code on the HPC, preferably using the terminal.

1. **Autolab** Write a Python program that writes "Hello world" to a file (e.g., called 'content.txt'). It should also print the same text to the screen.

   Answer: Autolab exercise - answer omitted.

2. Transfer your Python file to the HPC using scp, sftp, FileZilla, or other (see guide File Transfer to/from HPC on Learn).

   Answer: Assume the script is called `helloworld.py`. We can transfer it to HPC with the following SCP command:

   ```
   1  scp helloworld.py s123456@login.hpc.dtu.dk:helloworld.py
   ```

   If you already have directory you would like to store the file, you may upload directly to that directory with

   ```
   1  scp helloworld.py s123456@login.hpc.dtu.dk:Documents/02613/w01/
         helloworld.py
   ```

   assuming you have a directory named 'Documents/02613/w01/' in your home directory.

3. Connect to an HPC terminal (see guide Connecting to an HPC Terminal on Learn). If you are *not on a DTU network*, you may need to use a VPN or set up SSH keys - see the guide. If you used SSH, make sure you called `linuxsh` so you are *not* on a login node.

   Answer: With SSH, log on to the cluster with:

   ```
   1  ssh s123456@login.hpc.dtu.dk
   ```

Type in your password to log on. Then, run:

```
linuxsh
```

4. Create and/or navigate to your working directory (on the HPC) for this exercise (see guide Linux Terminal Cheat Sheet on Learn).

Answer: Let's assume we didn't already have a directory when we uploaded the file. The following three commands creates a new directory at `Documents/02613/w01', moves thehelloworld.py‘` file to the new directory and navigates to it:

```
mkdir -p Documents/02613/w01
mv helloworld.py -t Documents/02613/w01
cd Documents/02613/w01
```

If you already had the directory and uploaded the file directly to it, just run the `cd` command

```
cd Documents/02613/w01
```

5. Initialize the course conda environment (see guide Initializing the 02613 Conda Environment on Learn).

Answer: First, activate the shared Anaconda installation by running

```
source /dtu/projects/02613_2025/conda/conda_init.sh
```

Then, activate the conda environment by running

```
conda activate 02613
```

6. Run your Python program.

Answer: Run your program with

```
python helloworld.py
```

7. Transfer the generated file (e.g., `content.txt`) back to your local computer.

Answer: Exit your SSH connection (e.g., by running `exit` until you are back in your own terminal). Then, run

```
scp s123456@login.hpc.dtu.dk:Documents/02613/w01/helloworld.py
    helloworld.py
```

2. Job Scripts

For all scripts below, use `/bin/sleep 60` (or a longer period, like 100 or 120 seconds) as the command to run, and use 'hpc' as the queue name (except if stated otherwise).

1. **Autolab** Write a simple job script, like the one shown in the lectur and submit it.

Answer: Autolab exercise - Job script omitted.

a) Check the status with bstat and/or bjobs. Use 'man bjobs', to get information about the options or check the online documentation.

Answer: Running `bstat` just after submitted will print something like:

```
$ bstat
JOBID     USER    QUEUE   JOB_NAME  NALLOC STAT  START_TIME
       ELAPSED
19882525  patmjen hpc     sleeper        0 PEND       -
           0:00:00
```

We can find the status under the \texttt{STAT} column. We can see it is \texttt{PEND} for `pending` and there is no start time as it has not started. We can get the same information from `bjobs`:

```
$ bjobs
JOBID     USER    QUEUE   JOB_NAME   SLOTS STAT  START_TIME
   TIME_LEFT
19882525  patmjen hpc     sleeper      -   PEND       -
```

Once the, job has started, the output of `bstat` will change to:

```
1  $ bstat
2  JOBID     USER    QUEUE   JOB_NAME  NALLOC STAT  START_TIME
         ELAPSED
3  19882525  patmjen hpc     sleeper        1 RUN   Jan 10 16:39
       0:00:07
```

The status is now RUN for `running` and there is a start time. A similar change occurs for `bjobs`:

```
1  $ bjobs
2  JOBID     USER    QUEUE   JOB_NAME   SLOTS STAT  START_TIME
      TIME_LEFT
3  19882525  patmjen hpc     sleeper        1 RUN   Jan 10 16:39
      00:01:53 L
```

b) You can add a walltime limit to the script. Can you see that limit in the bstat or the bjobs output?

Answer: The `bstat` output does not include information about the wall time limit. The `bjobs` output includes the `TIME_LEFT` column, which tells us how time is left before we hit the wall time limit we set.

2. **Autolab** Write a job script that sends you notifications when the job starts and ends - see 'man bsub' for the details or check the online documentation. Take a look at the job summary, i.e. what information can you retrieve from that?

Answer: Autolab exercise - Job script omitted.

The most important part of the summary is the last part. It should say "Successfully completed" indicating the job finished as expected. Following that is a summary of the resource use. **Note:** If 'Run time' or 'Max Memory' differs significantly from the requested, consider changing it if it's a job you will be running again.

In the top, we may also see what directories were used as the home and working directories. This can be useful for debugging. Finally, in the middle part, out job script is repeated so we may see what we submitted.

a) To test, increase the period in the sleep command to be longer than the wall time limit, and submit the job again. What happens?

Answer: The job will start as before, but after the wall time limit is reached it will be terminated. In the job summary, instead of "Successfully completed." it will say something like "TERM_RUNLIMIT: job killed after reaching LSF run time limit."

---

3. The default `hpc` queue has nodes of different type, e.g. with different CPUs. The CPU type can be requested as a feature in a command script.

   a) **Autolab** Use the `nodestat` command to check which CPU types are available in the `hpc` queue, and then submit a job script that requests one of the types. See the Batch Jobs under LSF 10 webpage for information on how to do that.

   Answer: To see the nodes in the `hpc` queue with their CPU types, run:

   ```
   1  nodestat -f hpc
   ```

   Autolab exercise - Job script omitted.

   ---

   b) Add the necessary commands to your job script, to print the CPU type - and check in the job output that your job did indeed run on a node with the requested feature. Note: there is slight difference in the type request and the type variable: the variable contains a '-', while LSF uses a '_'.

   Answer: Add `lscpu` to the job script in order to print information on the CPU type. You can then check what was printed in 'sleeper_XXXXXX.out' after the job finishes.

The next exercises are a preparation for later weeks, where we want to submit multi-core jobs to the batch system:

4. **Autolab** Write a job script that requests 1 node and 4 cores.

   Answer: Autolab exercise - Job script omitted.

   ---

5. **Autolab** Write a job script, requesting 1 node and 16 cores. Does it run? If the job doesn't start, use 'bjobs -p' to check for the reason.

Answer: Autolab exercise - Job script omitted.

It will start, as the `hpc` queue includes nodes with processors that have more than 16 cores available.

---

6. **Autolab** Write a job script, requesting 1 node and 64 cores. Does it run? If the job doesn't start, use the `bjobs -p` command to check for the reason.

Answer: Autolab exercise - Job script omitted.

It will not start, as the 'hpc' queue does not includes nodes with processors that have 64 cores. Running `bjobs -p` will show something like:

```
1  JOBID      USER    STAT  QUEUE    FROM_HOST   EXEC_HOST  JOB_NAME
      SUBMIT_TIME
2  19882618   patmjen PEND  hpc      hpclogin2              sleeper
      Jan 10 16:33
3  Not enough job slot(s): 10 hosts;
```

The message `Not enough job slot(s)` is what indicates that no node has enough CPU cores available to run our job.

Clean up:

7. Check all your submitted jobs with 'bstat' again. If there are any left, that still are in status 'PEND', please remove them with 'bkill JOBID' (the JOBID is the number in the first column of the 'bstat'/'bjobs' output).

Answer: For example, assume the `bstat` output is:

```
1  JOBID      USER    QUEUE    JOB_NAME  NALLOC STAT  START_TIME
      ELAPSED
2  19882618   patmjen hpc      sleeper        0 PEND       -
      0:00:00
```

Then, run:

```
bkill 19882618
```

to kill the pending job.

Hints: to get the informations needed, use the 'man' command and take a look at the DTU Computing Center webpages https://www.hpc.dtu.dk/?page_id=2534